

NOAH

Autonomous Delivery Rover

System Overview & Operator Manual

A first-time operator's guide to the rover's hardware, radio controls, autonomous mission, docking and undocking, obstacle avoidance, sidewalk edge guidance, and on-board status displays.

Generated June 04, 2026

Platform: Raspberry Pi 5 (16 GB) • Node.js + Python vision

Contents

1. Safety First
2. System Overview
3. Electrical Components & Wiring
4. The RC Transmitter — Channel Map
5. Operating Modes: RC vs. Mission
6. Undocking
7. The Autonomous Mission
8. Docking (Return & Light-Seeking)
9. Object Avoidance
10. Sidewalk Edge Detection & Guidance
11. LCD Screen Messages
12. Typical Operating Sequences
13. Troubleshooting
14. Quick Reference — Key Settings

1 Safety First

Noah is a self-driving vehicle that can start moving on its own. Read this section before powering the rover on for the first time.

- Keep people, pets, and obstructions clear before arming, undocking, docking, or starting a mission.
- The RC transmitter is your master override. Putting the rover into **RC mode** (channel 11 high) immediately cancels an autonomous mission and returns manual control of steering and throttle.
- Always know where the throttle stick (CH3) is. In RC mode, centering it stops the wheels.
- If a dock or undock sequence is doing something unexpected, move the dock switch (CH7) to its **center** position — this halts dock/undock sequences.
- On any abnormal behavior: stop movement first, then diagnose.

Important

The rover must be armed on the Pixhawk flight controller before any radio stick or switch has an effect. The radio does not arm or disarm the vehicle — it selects modes and controls behavior once the Pixhawk is armed.

2 System Overview

Noah is an autonomous delivery rover built on a **Raspberry Pi 5 (16 GB)** running a Node.js application. Its job is simple to state: drive a route of GPS waypoints from its dock to a delivery point, release the package, and return home to re-dock — while avoiding obstacles and keeping itself on the sidewalk along the way.

The Raspberry Pi is the brain. It talks to a **Pixhawk flight controller** over a serial link for GPS, attitude, servo output, and the RC radio. A separate **Python vision program** processes the Intel RealSense depth camera and streams results back to the main application as JSON. Motor drivers, the steering servos, the package-delivery Arduino, the LiDAR, the IMU, the dock beacon sensor, and three LCD status screens all connect to the Pi as well.

How the pieces fit together

- **Sensing:** a Here+ RTK base sharpens GPS to centimeters; GPS + IMU give position and heading; RPLiDAR and the RealSense camera see obstacles; the camera also finds the sidewalk edge; an IRLock sensor finds the dock's IR beacon.
- **Thinking:** A control loop runs every **250 ms**. Each tick it decides where to steer, how fast to drive, and whether it must stop or go around something.
- **Acting:** Two ZLAC8015D drivers spin the four wheel motors; steering servos (driven through the Pixhawk) point the wheels; an Arduino runs the package-delivery mechanism.
- **Communicating:** Three LCD screens show a face, a device-health panel, and a live vision panel so an operator can read rover status at a glance.

The mission at a glance

Undock (drive down the dock ramp) → **Outbound** (follow GPS waypoints, hold the sidewalk edge) → **Deliver** (turn around, release package) → **Return** (retrace the route) → **Dock** (align to the recorded heading and home in on the IR beacon). Obstacle avoidance and emergency stop run the entire time.

3 Electrical Components & Wiring

Every device below connects to the Raspberry Pi 5. Serial devices use UART/USB ports; the IMU, dock beacon, and LCD screens share the I²C bus. The exact ports come from `setup.json`.

Serial / USB devices

Device	Role	Port	Baud	Protocol
Pixhawk flight controller	GPS, attitude, RC radio in, servo out	/dev/ttyAMA0	57,600	MAVLink
RPLiDAR	360° obstacle scanning	/dev/ttyAMA4	1,000,000	ultra_simple
RealSense camera link	Depth camera / vision subprocess	/dev/ttyAMA2	115,200	JSON
ZLAC8015D driver 1	Front-wheel motors (R-front, L-front)	/dev/ttySC0	115,200	Modbus RTU
ZLAC8015D driver 2	Rear-wheel motors (R-rear, L-rear)	/dev/ttySC1	115,200	Modbus RTU
Arduino	Package-delivery mechanism	/dev/ttyACM0	115,200	JSON

I²C bus devices (Bus 1)

Device	Address	Role
BNO055 IMU	0x28	Heading, pitch, roll (9-DOF fusion); polled every 50 ms
IRLock beacon sensor	0x54	Finds the dock's IR beacon for final docking approach
LCD 1 — Face	0x27	Animated mouth: smile = healthy, frown = a device is offline
LCD 2 — Status	0x26	Device health, battery voltage, satellites, heading
LCD 3 — Vision	0x25	Edge detection, avoidance state, LiDAR/vision/beacon status

Other hardware

- **Steering servos** (4): driven through the Pixhawk as servo outputs 11–14 (front-left, rear-left, front-right, rear-right).
- **RPLiDAR motor enable**: GPIO 23 turns the LiDAR spin motor on/off.
- **Delivery-type switch**: a GPIO input (BCM 17) selects the package-delivery style (arm vs. dump).
- **RealSense depth camera**: 640×480 @ ~15 fps, mounted 0.406 m above ground; processed by the Python vision program.
- **Battery monitoring**: read from the Pixhawk; a low-battery voice alert repeats when voltage falls below **20 V**.

Note

The four steering servos are commanded through the Pixhawk, not directly from the Pi. Wheel/turning geometry (e.g. wheelbase math) is separate from the physical wheel diameter, which is set as `voice.wheel_diameter_m` and used to convert encoder pulses into distance traveled.

GPS positioning & the Here+ RTK base (M8P)

Plain GPS is accurate to only a few meters — not enough to keep a rover centered on a sidewalk. To tighten that down to centimeters, Noah uses **RTK (Real-Time Kinematic)** positioning with a **Here+ RTK base station (u-blox M8P)**.

- **How it works:** the Here+ base sits on a known, fixed point and plugs into a laptop. It computes GPS correction data (RTCM messages) and that correction stream is relayed to the rover's GPS receiver.
- **What you get:** with corrections flowing, the rover's fix improves from a normal **3D** fix to an **RTK float** solution and then, once the carrier phase locks, an **RTK fixed** solution — centimeter-level position. This is what makes sidewalk edge guidance and accurate waypoint arrival possible.
- **Setup tips:** give the base a clear view of the sky and let it settle before driving; keep the correction link to the rover alive throughout the run; the closer the rover stays to the base, the faster it holds an RTK-fixed solution.

Watch the fix on the screen

The rover's GPS fix type and precision are shown live on the bottom row of LCD 3 (the vision screen) — for example `GPS:RTKfix HDOP:0.6`. Before starting a mission, confirm it reads **RTKflt** or ideally **RTKfix**, not just 3D. See §11 for the full breakdown.

4 The RC Transmitter — Channel Map

The radio receiver feeds the Pixhawk, which passes channel values to the rover. Every channel reading must fall in a valid range (roughly 800–2200 μ s); if the signal is lost or a channel reads invalid, the frame is rejected and RC-driven actions stop. Below, **Low**, **Center**, and **High** describe stick/switch position.

Ch	Control	Low	Center	High
CH1	Steering	Turn left	Straight	Turn right
CH3	Throttle	Reverse (to –max)	Stop	Forward (to +max)
CH4	Yaw / mode	Spin left in place	2-wheel steering mode	Spin right in place
CH5	Claw manual enable	Auto-delivery mode	—	Manual claw controls on
CH7	Dock / Undock	Dock (mission) / test-dock (RC)	Neutral · stops sequences	Undock sequence
CH8	Arm actuator	Lower	Idle	Raise
CH9	Avoidance toggle	Avoidance ON	—	Avoidance OFF
CH11	RC / Mission	Mission start / request	—	RC mode (cancels mission)
CH12	Belt motor	Reverse	Stop	Forward

Claw controls (CH5, CH8, CH12) and the telescope (CH2) only respond when the claw manual-enable switch (CH5) is high. With CH5 low the rover uses automatic package delivery during a mission.

How CH7 is armed

After power-up the dock switch (CH7) must pass through its center position once before it will act. This prevents a stale switch position from triggering an undock or dock the instant the rover boots.

Throttle is RC-mode only

CH3 throttle and CH1/CH4 steering only drive the rover in RC mode. During an autonomous mission the sticks are ignored — the rover steers itself. (An active voice command can also be cancelled by simply nudging a stick.)

5 Operating Modes: RC vs. Mission

The rover is always in one of two modes, selected by **channel 11**.

RC mode (CH11 high)

- You drive manually: CH1 steers, CH3 is throttle, CH4 spins the rover in place.
- Raising CH11 while a mission is running **immediately cancels the mission**, stops the motors, and re-centers the wheels. This is your override.

Mission mode (CH11 low)

- To start a mission, raise CH11 to arm it, then lower it. The high-to-low transition is what launches the mission.
- The 250 ms autonomous control loop begins, and the rover follows its GPS waypoints.
- If you request a mission while the rover is still undocking, the mission is queued and starts automatically the moment undock completes.
- At launch, a short grace window suppresses obstacle avoidance so the rover can pull cleanly off the dock before it starts reacting to nearby structures.

6 Undocking

*Undocking drives the rover forward off its dock ramp. It is triggered by raising the dock switch (**CH7 high**) and runs as a self-contained sequence; it does not by itself start a mission.*

What happens, step by step

- **Record start pose.** The rover saves its current GPS position, heading, and pitch as the dock reference.
- **Drive forward.** All four motors creep forward at low speed.
- **Detect the ramp.** The rover watches its pitch. A change of about 0.12 rad ($\sim 7^\circ$) from the dock reference means it is on the ramp. If no ramp is seen within ~ 5 seconds, it assumes flat ground and proceeds anyway.
- **Roll down and level out.** Travel continues until pitch returns to within ~ 0.07 rad ($\sim 4^\circ$) of level for at least a second — the ramp is behind it. If the IR beacon is visible, the rover steers to stay centered on it while descending.
- **Clear the dock.** It drives forward a little longer (a couple of seconds) to fully clear the structure, then stops and records its **undock position and heading**. These are reused later to return and re-dock.

Undock then mission

If you set CH11 low (mission request) during the undock, the mission auto-starts the moment undock completes — you don't need to time it perfectly.

7 The Autonomous Mission

Once running, the mission loop ticks every **250 ms**. Each tick it reads GPS and heading, checks for obstacles, decides how to steer and how fast to go, and watches for arrival at the next waypoint.

Following GPS waypoints

- Waypoints are uploaded to the rover (latitude/longitude, in order). The rover computes the bearing to the current waypoint and the difference from its current heading (the **yaw error**).
- A waypoint is considered **reached** when the rover's GPS sits within about half a meter of it for a few consecutive ticks (so GPS jitter can't cause false arrivals). Then it advances to the next waypoint.

Two ways to steer

- **2-wheel (Ackermann) steering** for small heading corrections: the front wheels angle and the rover drives forward, gently curving onto the bearing. Speed automatically eases off as the yaw error grows.
- **4-wheel spin-in-place** for big turns: when the yaw error exceeds about **20°** (`mission_yaw_start_deg`), the rover stops, points all four wheels, and rotates on the spot until it is roughly aligned, then resumes forward driving. Hysteresis around the threshold prevents it from flip-flopping between modes.

Delivering the package

- When the rover reaches the final waypoint, it turns to face back the way it came (toward the previous waypoint) so it is lined up for the return trip.
- It then runs the delivery mechanism via the Arduino. A 10-second fallback timer guarantees the mission proceeds even if the mechanism doesn't report completion.

Returning home

- The rover retraces its route — preferentially replaying a breadcrumb trail of points it recorded on the way out, otherwise reversing through the waypoint list.
- On the final approach it overrides the last waypoint with the exact **undock position** it recorded earlier, then aligns to the **undock heading** and hands off to docking.

The sidewalk-following gate

Camera-based sidewalk following starts **off** — this prevents false steering from driveways and roads near the dock. Reaching a waypoint whose route turn exceeds **90°** (`sidewalk_gate_turn_deg`) on the way **out** turns sidewalk following ON; passing back through that same waypoint on the **return** turns it OFF. Place one sharp (>90°) waypoint at the sidewalk entrance to mark the boundary. Obstacle avoidance, emergency stop, and LiDAR run regardless.

8 Docking (Return & Light-Seeking)

After the rover returns to its undock position and aligns to the saved undock heading, it docks. Whether docking is automatic or waits for your command depends on how the dock switch (CH7) was handled during the mission.

Automatic vs. manual dock authorization

- **Automatic:** if the dock switch was left untouched (or set low to authorize docking) during the mission, the rover docks on its own as soon as it returns and aligns.
- **Manual:** if you moved CH7 to **center** during the mission, the rover returns, aligns, and then **waits**. It will not dock until you explicitly command it by moving CH7 **low**.

Light-seeking final approach

- The IRLock sensor looks for the dock's IR beacon. If the beacon isn't visible, the rover spins slowly in place to search for it (giving up after about a minute if it never appears).
- Once the beacon is found, the rover backs toward the dock, steering to keep the beacon centered. Brief beacon dropouts are tolerated (it holds course for a couple of seconds before re-searching).
- It climbs the ramp — detected the same way as undocking, by watching pitch — drives forward a little at the top to seat itself, and records the final dock pose. Docking is complete.
- If the beacon is never available, a pitch-only fallback backs the rover straight up the ramp.

Center = stop

Moving CH7 to its center position at any time stops dock and undock sequences and holds the rover. Use it if a docking approach looks wrong.

9 Object Avoidance

Obstacle avoidance uses the RPLiDAR and the RealSense camera together. The LiDAR's sweep is divided into zones: the two **front zones (11 and 12)** tell the rover whether the path straight ahead is blocked, while **zones 1–10** around the rover are candidate directions to turn toward. The camera, mounted lower than the LiDAR, catches short obstacles the LiDAR can miss.

When it engages

- Avoidance is active only during a mission, with a valid RC link, and with the avoidance toggle enabled (**CH9 low**).
- It triggers on a blocked front arc — front LiDAR zones turning red, or the camera's front zones flagging an object. A short confirm delay (~300 ms) filters out momentary noise so the rover doesn't swerve at flickers.
- **First-leg commitment:** right after launch the rover ignores stale pre-start detections until it has committed to the first leg (driven roughly a meter), so the dock structure can't trigger an immediate avoidance.

How it gets around

- It picks the nearest clear (green) zone — biased toward the direction of the next waypoint — and turns that way. A clear zone on the right turns it right; on the left, left.
- Once the front clears and stays clear, it creeps forward until the corridor is reliably open (about a meter of clearance), then resumes normal navigation.
- If no clear zone exists at all, the rover stops and holds rather than guessing.

Emergency stop & blocked-path fallback

- A high-threat object within about **1 m** (`object_emergency_stop_m`) directly in the path triggers an immediate stop.
- If the path stays blocked for **10 s** (`rs_block_timeout_ms`), or avoidance runs out of options (its ~30 s timeout), the rover gives up on reaching the next waypoint and performs a fallback delivery at its last confirmed position rather than sitting forever.

Disabling avoidance

Setting CH9 high turns off LiDAR-based avoidance. The camera's emergency-stop behavior and GPS navigation continue. Only disable avoidance when you have a clear reason to.

10 Sidewalk Edge Detection & Guidance

GPS gets the rover to the right block; the camera keeps it on the sidewalk. A Python vision program processes the RealSense depth camera and streams results to the main application as JSON. Its key output for steering is the **sidewalk edge about two feet ahead** — not the whole-path centerline.

Finding the edge

- The vision program scans a band of the image near the lookahead distance (`edge_lookahead_m` $\approx$ 0.61 m / 2 ft) and finds the left and right sidewalk edges using a blend of cues: concrete color, depth gradient, and the signed drop-off at the curb. Each edge comes with a confidence score.
- The rover aims to hold itself about **1.5 ft** (`edge_side_offset_m` $\approx$ 0.46 m) off the chosen edge. If both edges are visible, it uses the higher-confidence one; left-only $\rightarrow$ stay 1.5 ft to its right; right-only $\rightarrow$ stay 1.5 ft to its left.
- The result is emitted as an angle to the desired track position (`x_angle_deg`), which feeds the existing 2-wheel steering layer — so edge guidance is just a steering nudge layered on top of GPS navigation.

When the edge disappears

- Hysteresis keeps the rover committed to one edge instead of oscillating when both are visible and confidences wobble.
- If edge confidence drops below threshold (no edge in view), the correction **latches and then fades** over a short window, after which the rover falls back to GPS-only navigation.
- A camera-mount sign (`correction_direction`) flips the correction direction if the camera is mounted such that corrections would otherwise go the wrong way.

Edge guidance is gated

Sidewalk edge steering only acts between the $>90^\circ$ gate waypoints (see §7). On driveways, roads, and near the dock it is suppressed so the rover doesn't chase a false edge.

11 LCD Screen Messages

Three 20×4 LCD screens report rover status. Screen 1 (the face) updates about twice a second; Screens 2 and 3 refresh about once a second.

LCD 1 — Face (address 0x27)

- **Smile:** all monitored devices are connected and healthy.
- **Frown:** at least one major subsystem (LiDAR, Pixhawk, motor drivers, Arduino, RealSense, IMU, or IRLock) is not connected.
- **Talking mouth:** animates while the rover is speaking (voice output).

LCD 2 — Device Status (address 0x26)

Shows each device as OK (connected) or - - (offline), plus live telemetry:

Field	Meaning
LIDAR / PIX	RPLiDAR and Pixhawk connection status
WAVE / ARD	Motor drivers (both) and Arduino connection status
RS / IMU / IRL	RealSense, IMU connection; IRLock 1 = connected, 0 = not
BAT	Battery voltage, e.g. 24.5V (low-battery alert below 20 V)
S	Visible GPS satellites (-- if unknown)
H	Heading in degrees (0–359)

LCD 3 — Vision, Avoidance & GPS (address 0x25)

Rows 0–1 report the left and right sidewalk edges; row 2 reports avoidance and sensor status; row 3 reports the GPS fix and precision.

Field	Meaning
EL / ER	Edge Left / Edge Right seen this frame: 1 = yes, 0 = no
C	Edge confidence (1–99), or -- when no edge
X	Edge lateral position from rover center (m), or ----
Y	Edge forward distance ahead (m), or ----
AV	Avoidance: CLR = clear · BLK = blocked · TMO = avoidance timed out
L	LiDAR front arc: G green · Y yellow · R red
V	RealSense front object zone: G green · Y yellow · R red
IRL	IRLock beacon: 1 = fresh/visible, 0 = not
GPS	GPS fix type (see RTK note below)
HDOP	Horizontal dilution of precision — lower is better; -- if unknown

Row 3 reads, for example, `GPS:RTKfix HDOP:0.6`. The GPS fix type can be: `NOGPS`, `NOFIX`, `2D`, `3D`, `DGPS`, `RTKflt` (RTK float), `RTKfix` (RTK fixed), `STATIC`, or `PPP`.

Reading green/yellow/red

Green means clear, yellow means something is detected at moderate range, red means an obstacle is close. If a sensor's data goes stale, its status reverts to green/clear.

12 Typical Operating Sequences

A. Start a delivery mission (with undock)

- Confirm the area is clear and the rover is armed on the Pixhawk.
- Cycle CH7 through center once (arms the dock switch), then set **CH7 high** to undock.
- Set **CH11 low** during or after undock to start the mission. It auto-starts when undock finishes.
- The rover follows its waypoints, delivers, and returns automatically.

B. Require manual permission before docking

- During the mission, flip **CH7 to center** once.
- The rover returns and aligns to the dock heading, then holds and waits.
- When you're ready, set **CH7 low** to authorize docking. The rover homes in on the beacon and docks.

C. Take over manually at any time

- Set **CH11 high**. The mission cancels, motors stop, wheels center.
- Drive with CH1 (steer), CH3 (throttle), CH4 (spin in place).

13 Troubleshooting

Symptom	Likely cause / what to check
Mission won't start	Confirm the rover is armed on the Pixhawk and RC channels read valid. CH11 needs a clean high-to-low transition. If undocking, the mission is queued until undock finishes.
Rover waits near the dock and won't dock	Manual dock authorization is required — you likely set CH7 to center during the mission. Set CH7 low to authorize docking.
Avoidance not reacting	Make sure a mission is active and CH9 is set low (avoidance enabled). Avoidance is also held off briefly at launch and until the first-leg commitment.
Rover swerves on a driveway / chases a false edge	Sidewalk following may be on outside the intended zone. Confirm a >90° gate waypoint is placed at the true sidewalk entrance.
Face screen shows a frown	A device is offline. Check LCD 2 for which one reads -- and inspect that device's cable/port.
Steering corrections go the wrong way	The camera-mount sign may be inverted — check <code>correction_direction</code> in the RealSense settings.
No GPS / satellites show --	Wait for GPS lock outdoors with a clear sky view; the mission rejects zero/invalid GPS to avoid false arrivals.
GPS won't reach RTKfix (stuck at 3D)	Check the Here+ base has power, a clear sky view, and that its correction link to the rover is live. Let the base settle; keep the rover nearer the base. Watch the fix on LCD 3 row 3.
Low-battery voice alert repeating	Battery is below 20 V. Stop, recover the rover, and charge.

14 Quick Reference — Key Settings

These tunable values live in `setup.json`. Defaults shown; adjust with care. (Per project rule, any change to `setup.json` must also be mirrored in `setup_example.json`.)

Setting	Default	What it controls
<code>mission_yaw_start_deg</code>	20°	Yaw error above which the rover switches to 4-wheel spin-in-place
<code>sidewalk_gate_turn_deg</code>	90°	Route turn at a waypoint that toggles sidewalk following
<code>rs_block_timeout_ms</code>	10,000 ms	How long a blocked path persists before fallback delivery
<code>object_emergency_stop_m</code>	1.0 m	Distance at which a high-threat object triggers an emergency stop
<code>edge_lookahead_m</code>	0.6096 m	How far ahead the camera reads the sidewalk edge (2 ft)
<code>edge_side_offset_m</code>	0.4572 m	Lateral gap the rover holds off the edge (1.5 ft)
<code>camera_height_m</code>	0.406 m	RealSense mount height above ground
<code>low_battery_voltage</code>	20 V	Battery voltage that triggers the low-battery alert
<code>wheel_diameter_m</code>	0.2413 m	Physical wheel diameter (9.5 in); converts encoder pulses to distance
<code>throttle_percentage</code>	25	Base throttle scaling for motor commands

End of manual. For deeper architecture notes see `CLAUDE.md` and the `.claude/memory/` folder in the project repository.